

 SATYACHECK — AI FAKE NEWS DEFENSE SYSTEM

Backend **Development** Role Documentation

Complete Node.js + Express backend walkthrough with real project code,
covering every topic from the course syllabus

PROJECT

SatyaCheck

ROLE

Backend Developer

STACK

Node.js · Express · REST API

DATE

September 2026



BACKEND ROLE · FULL EXPLANATION



Table of Contents

Topics covered in this document — mapped to syllabus

1	Node.js Modules	Built-In Custom	path · fs · events · ESM vs CommonJS
2	HTTP Basics	Core	Basic Server · GET · POST · Static Files · Endpoints
3	Express Framework	Main Tool	Setup · dotenv · Middleware · Static · Routes · Sessions
4	External Routers	Architecture	Splitting routes · Dynamic Routing · Route params
5	Authentication	Security	Register · Login · Password Hashing · Token · Session
6	CORS & Frontend-Backend Connection		Cross-origin requests · Proxy · API communication
7	Multer — File Uploads		Uploading images to the backend
8	All API Endpoints in SatyaCheck		Complete route map of the project backend
9	Project Folder Structure		How the backend is organized



How to Read This Document

Every section shows the CONCEPT first (what it is, why we use it), then the ACTUAL CODE from the SatyaCheck project so you can explain exactly what each line does to your teacher.

1

Node.js Modules

Built-in · Custom · ESM vs CommonJS

ESM vs CommonJS — Two Module Systems

Node.js supports two ways to share code between files. SatyaCheck uses **ESM (ECMAScript Modules)** — the modern standard.

Feature	CommonJS (old)	ESM — Used in SatyaCheck 
Syntax to import	<code>const x = require('./file')</code>	<code>import x from './file.js'</code>
Syntax to export	<code>module.exports = x</code>	<code>export default x</code> or <code>export { x }</code>
Declared in package.json	"type": "commonjs" (default)	"type": "module"
File extension	.js or .cjs	.js or .mjs
Top-level await	 Not supported	 Supported

```
ARTIFACTS/API-SERVER/PACKAGE.JSON — ESM DECLARATION
{
  "name": "satyacheck-api",
  "type": "module", // ← This enables ESM for all .js files
  "main": "src/index.js"
}
```

Built-in Module: path

Used to work with file and directory paths safely across Windows and Linux.

```
ARTIFACTS/API-SERVER/SRC/APP.JS — PATH MODULE USAGE

import path from "path"; // Built-in — no install needed
import { fileURLToPath } from "url"; // Needed in ESM to get __dirname

// In ESM, __dirname is not available by default, so we recreate it:
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Build the path to frontend's built files
const staticPath = path.join(__dirname, "../../frontend-js/dist");
```

```
// Serve the built React app as static files in production
app.use(express.static(staticPath));
```

Built-in Module: fs (File System)

Used to read and write files on disk. SatyaCheck uses it to save user data and analysis results as JSON files when a database is not connected.

```
ARTIFACTS/API-SERVER/SRC/ROUTES/AUTH.JS - FS MODULE USAGE

import fs    from "node:fs";    // Built-in file system module
import path  from "node:path";  // Built-in path module

// The file path where users will be stored
const USERS_FILE = path.resolve(process.cwd(), "users-storage.json");

// READ: Load all users from file
function loadUsers() {
  if (fs.existsSync(USERS_FILE)) {
    return JSON.parse(fs.readFileSync(USERS_FILE, "utf-8"));
  }
  return [];
}

// WRITE: Save users back to file
function saveUsers(users) {
  fs.writeFileSync(USERS_FILE, JSON.stringify(users, null, 2), "utf-8");
}
```

Built-in Module: crypto

Used for security operations like hashing passwords so they are never stored in plain text.

```
AUTH.JS - CRYPTO FOR PASSWORD HASHING

import crypto from "node:crypto";

// Hash a password - never store plain text passwords!
function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString("hex");
  const key  = crypto.scryptSync(password, salt, 64);
  return `${salt}:${key.toString("hex")}`;
}

// Result stored in DB: "a3f5c8...abc123:def456..." - unreadable by humans!

// Also generates secure random session tokens
const token = crypto.randomBytes(32).toString("hex");
```

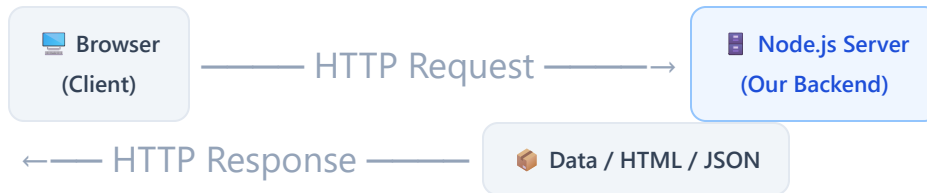

2

HTTP — HyperText Transfer Protocol

How the browser talks to the server

What is HTTP?

HTTP is the language the browser (client) uses to communicate with the server. Every time you open a website or submit a form, an HTTP request is made.



Basic HTTP Server using Node's Built-in http module

This is the simplest possible server — no Express yet. Pure Node.js.

CONCEPT — RAW HTTP MODULE (NOT IN PROJECT, BUT TAUGHT IN SYLLABUS)

```
import http from "http";

const server = http.createServer((req, res) => {
  // req = the incoming request from browser
  // res = what we send back

  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Hello from SatyaCheck!</h1>");
  } else {
    res.writeHead(404);
    res.end("Not Found");
  }
});

server.listen(3000, () => console.log("Server on port 3000"));
```



Why not use raw http in our project?

The raw http module is powerful but very low-level — you have to manually handle every URL. In SatyaCheck, we use

Express

which wraps http and gives us clean, organized routing. Express is built on top of http internally.

HTTP Methods Explained

Method	Used For	Example in SatyaCheck	Has Body?
GET	Fetch data from server	GET /api/healthz, GET /api/history	✗ No
POST	Send data to server	POST /api/auth/login, POST /api/analyze	✓ Yes (JSON)
PUT	Update existing data	Not yet used	✓ Yes
DELETE	Delete data	Not yet used	✗ No

HTTP Status Codes Used in SatyaCheck

Code	Meaning	When We Use It
200	OK — Success	Successful login, data fetched
201	Created	New account registered successfully
400	Bad Request	Missing name / email in form
401	Unauthorized	Wrong password, invalid token
409	Conflict	Email already registered
500	Server Error	Something crashed on our backend

Package Installation

TERMINAL — PACKAGES INSTALLED IN SATYACHECK BACKEND

```
# Install all required packages
npm install express      # Web framework (replaces raw http)
npm install cors         # Allow frontend to talk to backend
npm install pino pino-http # Professional logging
npm install openai       # AI model integration (Ollama)

# Dev tools
npm install --save-dev nodemon # Auto-restart on file change
```

Environment Variables with .env file

We never hardcode sensitive values like passwords or ports in code. Instead, they go in a `.env` file which is **never uploaded to GitHub**.

.ENV — SATYACHECK ENVIRONMENT VARIABLES

```
# Server config
PORT=3000
NODE_ENV=development

# AI model (Ollama running locally)
OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
OLLAMA_MODEL=llama3.2:1b

# Database (optional)
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/satyacheck
```

INDEX.JS — READING THE PORT FROM ENVIRONMENT

```
// Node 20+ loads .env automatically with --env-file flag
// Run: node --env-file=".env" artifacts/api-server/src/index.js

const port = Number(process.env.PORT || 3000);
// process.env reads from the .env file
// Falls back to 3000 if PORT is not set

app.listen(port, () => console.log(`Server running on port ${port}`));
```

Express App Setup & Middleware

Middleware = functions that run on EVERY request before it reaches the route handler. Think of it like airport security — every passenger goes through it.

```
ARTIFACTS/API-SERVER/SRC/APP.JS — FULL EXPRESS SETUP

import express    from "express";
import cors       from "cors";
import pinoHttp   from "pino-http";
import path       from "path";
import router     from "../routes/index.js";

const app = express(); // Create the app

// — MIDDLEWARE (runs on every request) —
app.use(cors()); // Allow cross-origin requests
app.use(express.json({ limit: "50mb" })); // Parse JSON request bodies
app.use(express.urlencoded({ extended: true })); // Parse form data
app.use(pinoHttp({ logger })); // Log every request

// — API ROUTES —
app.use("/api", router); // ALL routes start with /api

// — STATIC FILE SERVING (PRODUCTION ONLY) —
if (process.env.NODE_ENV === "production") {
  const staticPath = path.join(__dirname, "../frontend-js/dist");
  app.use(express.static(staticPath)); // Serve HTML, CSS, JS files

  // — SPLAT / FALLBACK ROUTE — send index.html for all non-API paths
  app.get(/^(?!\/api).*/, (_req, res) => {
    res.sendFile(path.join(staticPath, "index.html"));
  });
}

export default app;
```



What is the Splat/Fallback Route?

The regex `/^(?!\/api).*/` means: "match ANY path that does NOT start with `/api`". This is the SPA (Single Page Application) fallback — it sends React's `index.html` for every unknown route so React Router can handle the navigation on the client side.

4

External Routers & Dynamic Routing

Splitting routes into separate files for clean code

Why External Routers?

Instead of putting ALL routes in one giant file, we split them by feature. SatyaCheck has 7 separate router files — each handling one area of the backend.



The Route Aggregator File

ARTIFACTS/API-SERVER/SRC/ROUTES/INDEX.JS — IMPORTS & MOUNTS ALL ROUTERS

```
import { Router } from "express";

// Import every feature router
import healthRouter from "./health.js";
import analyzeRouter from "./analyze.js";
import chatRouter from "./chat.js";
import credibilityRouter from "./credibility.js";
import trendingRouter from "./trending.js";
import quizRouter from "./quiz.js";
import authRouter from "./auth.js";
import usersRouter from "./users.js";

const router = Router();

// Mount each router — they handle their own sub-paths
router.use(healthRouter);
router.use(analyzeRouter);
router.use(chatRouter);
router.use(credibilityRouter);
router.use(trendingRouter);
router.use(quizRouter);
router.use(authRouter);
router.use(usersRouter);

export default router;
```

Dynamic Routing — URL Parameters

Dynamic routes use `:param` in the URL. The value after the colon is extracted from the URL.

■ CONCEPT OF DYNAMIC ROUTES (USED IN AUTH.JS PATTERN)

```
const router = Router();

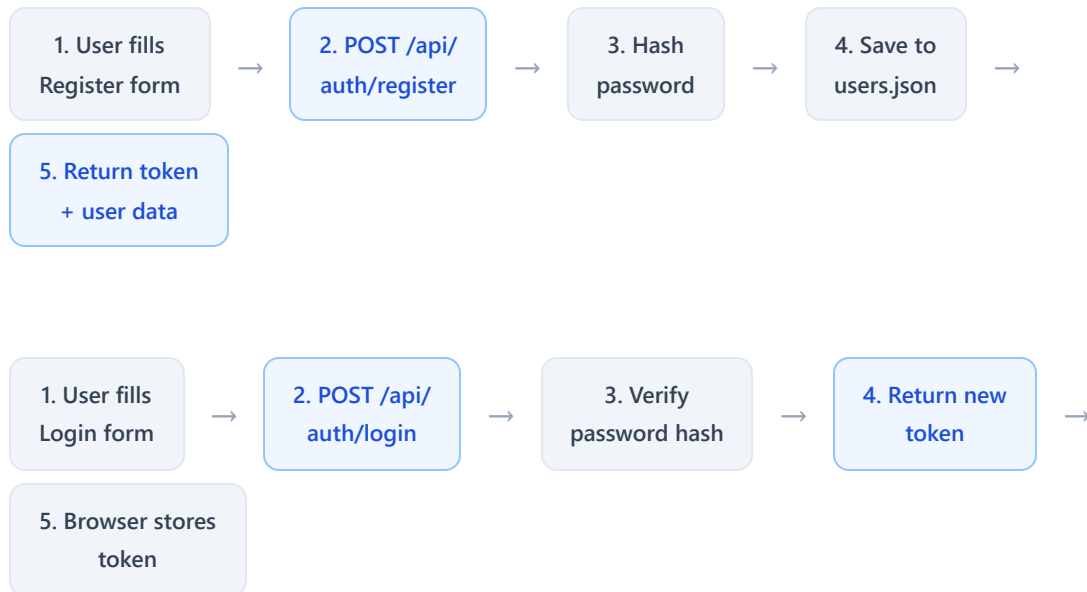
// Static route - exact URL match
router.get("/users", (req, res) => {
  res.json({ message: "All users" });
});

// Dynamic route - :id is a URL parameter
router.get("/users/:id", (req, res) => {
  const userId = req.params.id; // Extracted from URL
  // GET /api/users/42 → userId = "42"
  // GET /api/users/abc → userId = "abc"
  res.json({ userId });
});

// Multiple dynamic segments
router.get("/users/:id/posts/:postId", (req, res) => {
  const { id, postId } = req.params;
  res.json({ id, postId });
});

// Query string parameters (?key=value)
router.get("/search", (req, res) => {
  const q = req.query.q; // GET /api/search?q=fake+news
  res.json({ query: q });
});
```

How Authentication Works in SatyaCheck



Register Endpoint — POST /api/auth/register

```
ARTIFACTS/API-SERVER/SRC/ROUTES/AUTH.JS

router.post("/auth/register", (req, res) => {
  const { name, email, password } = req.body; // Data from form/frontend

  // 1. Validate input
  if (!name || !email || !password) {
    return res.status(400).json({ message: "All fields are required." });
  }
  if (password.length < 6) {
    return res.status(400).json({ message: "Password min 6 characters." });
  }

  // 2. Check if email already registered
  const users = loadUsers();
  const exists = users.find(u => u.email === email);
  if (exists) {
    return res.status(409).json({ message: "Email already registered." });
  }

  // 3. Hash the password (NEVER store plain text)
  const passwordHash = hashPassword(password);
```

```
// 4. Create and save the new user
const newUser = {
  id: `usr_${Date.now()}`,
  name, email, passwordHash,
  token: crypto.randomBytes(32).toString("hex"),
  createdAt: new Date().toISOString(),
};
users.unshift(newUser);
saveUsers(users);

// 5. Return success (without the passwordHash!)
const { passwordHash: _, ...safeUser } = newUser;
return res.status(201).json({
  success: true,
  user: safeUser,
  token: newUser.token,
});
});
```

Protecting Routes with Auth Middleware

```
AUTH.JS - GET /API/AUTH/ME - VALIDATES TOKEN

router.get("/auth/me", (req, res) => {
  // Browser sends: Authorization: Bearer <token>
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Unauthorized" });
  }

  const token = authHeader.replace("Bearer ", "");
  const users = loadUsers();
  const user = users.find(u => u.token === token);

  if (!user) {
    return res.status(401).json({ message: "Invalid token" });
  }

  return res.json({ user: sanitizeUser(user) });
});
```

CORS — Cross-Origin Resource Sharing

By default, browsers block requests from one domain to another (security feature). CORS tells the browser: *"It's OK — this server allows requests from the frontend."*



The Problem Without CORS:

Frontend runs on `http://localhost:5173` — Backend runs on `http://localhost:3000`. These are different "origins". Without CORS, the browser blocks all API calls and you get: `Access to fetch at '...' has been blocked by CORS policy`

APP.JS — CORS SETUP

```
import cors from "cors";

// Simple: allow ALL origins (fine for development)
app.use(cors());

// Production: only allow your specific frontend URL
app.use(cors({
  origin: ["http://localhost:5173", "https://satyacheck.vercel.app"],
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true, // Allow cookies/auth headers
}));
```

FRONTEND-JS/VITE.CONFIG.JS — VITE PROXY (ALTERNATIVE TO CORS IN DEV)

```
// In development, Vite acts as a proxy.
// Any /api call from the browser goes through Vite → backend.
// This means SAME origin, so no CORS needed in dev!
server: {
  proxy: {
    "/api": {
      target: "http://localhost:3000", // Forward to Express
      changeOrigin: true,
    },
  },
},
```

Multer — File Upload Middleware

Multer handles multipart/form-data — the format used to upload files (images, PDFs, etc.) from HTML forms or the frontend.

■ CONCEPT — HOW MULTER WORKS (READY TO ADD TO SATYACHECK FOR IMAGE ANALYSIS)

```
import multer from "multer"; // npm install multer

// Configure where and how files are stored
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads/"),
  filename: (req, file, cb) => cb(null, `${Date.now()}_${file.originalname}`),
});

const upload = multer({
  storage,
  limits: { fileSize: 10 * 1024 * 1024 }, // 10 MB max
});

// Route that accepts a file upload
// "image" matches the field name in the form
router.post("/analyze/image", upload.single("image"), (req, res) => {
  const file = req.file; // The uploaded file info
  const body = req.body; // Other form fields

  // file.path = where it was saved on disk
  // file.mimetype = "image/jpeg" etc.
  res.json({ saved: file.path });
});
```

Complete API Endpoint Map — SatyaCheck Backend

METHOD	ENDPOINT	WHAT IT DOES	FILE
GET	/api/healthz	Check if server is running	health.js
POST	/api/analyze	Run AI fact-check on a news claim	analyze.js
GET	/api/history	Get all past fact-checks	analyze.js
GET	/api/stats	Get analysis statistics	analyze.js
POST	/api/chat	SSE streaming AI chat	chat.js
GET	/api/credibility	Get news source credibility score	credibility.js
GET	/api/trending	Get trending misinformation stories	trending.js
GET	/api/quiz	Get a media literacy quiz	quiz.js

POST	/api/auth/register	Create new user account	auth.js
POST	/api/auth/login	Sign in with email + password	auth.js
POST	/api/auth/google	Sign in with Google OAuth	auth.js
GET	/api/auth/me	Get current user from token	auth.js
POST	/api/auth/forgot-password	Request password reset	auth.js
POST	/api/users/login	Log a user login event	users.js
GET	/api/auth/users	Admin: list all registered users	auth.js

7

Project Folder Structure

How the entire SatyaCheck backend is organized

COMPLETE BACKEND STRUCTURE

```
Fake-News-Defense/
├── artifacts/
│   └── api-server/
│       ├── package.json    ← Express dependencies
│       └── src/
│           ├── index.js    ← Entry point – starts server on PORT
│           ├── app.js      ← Express app setup – middleware + routes
│           ├── lib/
│           │   ├── logger.js ← Pino Logger configuration
│           │   └── scraper.js ← URL scraping utility (fetch article text)
│           └── routes/
│               ├── index.js ← Aggregates all routers
│               ├── health.js ← GET /api/healthz
│               ├── analyze.js ← POST /api/analyze (AI fact-check)
│               ├── chat.js   ← POST /api/chat (SSE streaming)
│               ├── credibility.js ← GET /api/credibility
│               ├── trending.js ← GET /api/trending
│               ├── quiz.js   ← GET /api/quiz
│               ├── auth.js   ← Register · Login · Google · Token
│               └── users.js  ← User audit log
├── frontend-js/           ← React frontend (separate)
├── .env                   ← Environment variables (NOT on GitHub)
├── users-storage.json     ← User database (JSON flat file)
└── analyses-storage.json  ← Analysis history (JSON flat file)
```

Syllabus Checklist — What Our Project Covers

Syllabus Topic	Status in SatyaCheck	File / Location
Built-in modules (path, fs, crypto)	✓ Used	auth.js, analyze.js, app.js
ESM (import/export)	✓ Used throughout	package.json "type":"module"
HTTP Basic Server	✓ Express wraps http	index.js → app.js
GET and POST endpoints	✓ 15 endpoints	All routes/*.js files
Static File Handling	✓ express.static	app.js (production block)
Splat/Fallback Route	✓ SPA fallback regex	app.js line 43

dotenv / Env Variables	✓ --env-file flag	.env file
Package install (express, dotenv)	✓ Installed	api-server/package.json
External Router	✓ 7 route files	routes/*.js
Dynamic Routing (:param)	✓ Used in auth	auth.js
Authentication	✓ Full system	auth.js — register/login/token
CORS (Frontend-Backend)	✓ cors middleware	app.js
Multer (file upload)	⚡ Ready to add	Can add to analyze.js
Session Auth	⚡ Token-based (similar)	auth.js — token in localStorage
MongoDB	⚡ Currently JSON/PostgreSQL	Can migrate to MongoDB



How to Run the Backend

```
node --env-file=".env" artifacts/api-server/src/index.js
```

Server starts on

http://localhost:3000

— test it at

http://localhost:3000/api/healthz



SatyaCheck Backend — Complete

Node.js ESM · Express 5 · Built-in Modules · REST API · Authentication · CORS · External Routers · Dynamic Routes

Project by Shivam Rana · Backend Role Documentation